

Renewably CRM — API Reference

For Internal Developers | Version 1.0 | April 2026

1. Overview & Architecture

Renewably CRM is a Next.js 16 App Router application deployed at renewably.ie, providing a comprehensive platform for managing solar energy sales, installer partnerships, and customer relationships. The system comprises 14 modules covering contacts, companies, deals, pipeline management, invoicing, proposals, tasks, meetings, activities, notes, workflows, reports, calendar integration, and billing.

The frontend features a dark theme with a #080808 background and #F3D840 yellow accent color. The application uses Prisma ORM with SQLite as the current database backend (migration to Supabase/PostgreSQL is planned). Authentication is session-based using bcryptjs for password hashing with HTTP-only cookie sessions.

Transaction emails are sent via the Postmark API using native fetch (no SDK dependency). Redis (via ioredis) provides caching and rate limiting with an in-memory fallback for environments without Redis. Billing is handled through Stripe with subscription checkout and customer portal integration. The AI chat feature uses the z-ai-web-dev-sdk.

2. Environment Variables

The following environment variables are used by the CRM system. Configure them in `.env.local` for development or in your hosting provider's environment settings for production.

Variable	Required	Description
DATABASE_URL	Yes	Database connection string (SQLite: file:./dev.db or Postgres for Supabase)
POSTMARK_SERVER_TOKEN	Yes	Postmark API server token for transactional email
FROM_EMAIL	No	Sender email address (default: hello@renewably.ie)
REDIS_URL	No	Redis connection URL (default: redis://localhost:6379)
STRIPE_SECRET_KEY	No	Stripe secret key for billing integration
STRIPE_WEBHOOK_SECRET	No	Stripe webhook signature verification secret
STRIPE_PRICE_STARTER	No	Stripe Price ID for Starter plan
STRIPE_PRICE_PRO	No	Stripe Price ID for Pro plan

Variable	Required	Description
STRIPE_PRICE_ENTERPRISE	No	Stripe Price ID for Enterprise plan
LOG_LEVEL	No	Logging level: debug/info/warn/error (default: info)
NODE_ENV	No	Environment: development/production

3. Database Schema (Prisma Models)

The CRM defines 7 Prisma models. All models use CUID primary keys (String, `@id @default(cuid())`) and automatic timestamps via `@updatedAt`.

User

Field	Type	Constraints	Notes
id	String	<code>@id @default(cuid())</code>	Primary key
email	String	<code>@unique</code>	Login email
passwordHash	String	—	bcryptjs hashed password
name	String	—	Display name
role	String	<code>@default("admin")</code>	admin manager user
avatar	String	Optional	Avatar URL
phone	String	Optional	Phone number
isActive	Boolean	<code>@default(true)</code>	Soft-disable flag
lastLoginAt	DateTime	Optional	Last successful login
createdAt	DateTime	<code>@default(now())</code>	Record creation
updatedAt	DateTime	<code>@updatedAt</code>	Auto-updated

Session

Field	Type	Constraints	Notes
id	String	<code>@id @default(cuid())</code>	Primary key
userId	String	<code>@relation</code>	FK → User.id
token	String	<code>@unique</code>	HTTP-only cookie value
expiresAt	DateTime	—	7-day expiry

Field	Type	Constraints	Notes
createdAt	DateTime	@default(now())	Record creation

Company

Field	Type	Constraints	Notes
id	String	@id @default(cuid())	Primary key
name	String	—	Company name
counties	String	Optional	Comma-separated county list
seaiReg	String	Optional	SEAI registration number
teamSize	Int	Optional	Number of employees
installsPerYear	Int	Optional	Annual installation count
status	String	@default("prospect")	prospect active inactive churned
website	String	Optional	Company website URL
notes	String	Optional	Free-text notes
createdAt	DateTime	@default(now())	Record creation
updatedAt	DateTime	@updatedAt	Auto-updated

Contact

Field	Type	Constraints	Notes
id	String	@id @default(cuid())	Primary key
companyId	String	@relation	FK → Company.id
name	String	—	Contact name
email	String	—	Contact email
phone	String	Optional	Phone number
role	String	Optional	Job title / role
isDecisionMaker	Boolean	Optional	Flag for decision authority
notes	String	Optional	Free-text notes
createdAt	DateTime	@default(now())	Record creation
updatedAt	DateTime	@updatedAt	Auto-updated

Deal

Field	Type	Constraints	Notes
id	String	@id @default(cuid())	Primary key
companyId	String	@relation	FK → Company.id
product	String	—	solarpilot ai_workforce both
mrr	Float	Optional	Monthly recurring revenue
setupFee	Float	Optional	One-time setup fee
stage	String	—	8-stage pipeline (see §5)
qualifiedAnswers	Json	Optional	Lead qualification data
demoOutcome	String	Optional	Demo call result
closeReason	String	Optional	Won/lost reason
assignedToId	String	Optional	FK → User.id
value	Float	Optional	Deal value (EUR)
notes	String	Optional	Free-text notes
createdAt	DateTime	@default(now())	Record creation
updatedAt	DateTime	@updatedAt	Auto-updated

Deal stages (in order): new → qualified → demo_scheduled → demo_completed → proposal_sent → negotiation → closed_won → closed_lost

DealActivity

Field	Type	Constraints	Notes
id	String	@id @default(cuid())	Primary key
dealId	String	@relation	FK → Deal.id
userId	String	@relation	FK → User.id
type	String	—	call email demo proposal note
title	String	—	Activity title
content	String	Optional	Activity details
createdAt	DateTime	@default(now())	Record creation

Onboarding

Field	Type	Constraints	Notes
id	String	@id @default(cuid())	Primary key

Field	Type	Constraints	Notes
companyId	String	@unique @relation	FK → Company.id (1:1)
solarpilotProgress	Int	@default(0)	Progress 0–100
aiWorkforceProgress	Int	@default(0)	Progress 0–100
solarpilotSteps	Json	Optional	Step completion data
aiWorkforceSteps	Json	Optional	Step completion data
startedAt	DateTime	Optional	Onboarding start time
completedAt	DateTime	Optional	Onboarding completion
createdAt	DateTime	@default(now())	Record creation
updatedAt	DateTime	@updatedAt	Auto-updated

4. Authentication API

Authentication uses bcryptjs password hashing with HTTP-only cookie-based sessions. All CRM endpoints require a valid session cookie (crm_session, 7-day expiry). Auth middleware is provided in `src/lib/crm-auth.ts`.

POST /api/crm/auth/login

Description: Authenticates a user and creates a session. Rate limited to 5 attempts per minute per IP address.

Request body:

```
{ "email": "user@renewably.ie", "password": "plaintext-password" }
```

Response (200):

```
{ "user": { "id": "clxyz...", "email": "user@renewably.ie", "name": "John Doe", "role": "admin", "avatar": null, "phone": null } }
```

Sets HTTP-only cookie: `crm_session` with 7-day max-age.

Error (401): { "error": "Invalid email or password" }

POST /api/crm/auth/logout

Description: Clears the `crm_session` cookie and invalidates the server-side session.

Response (200): { "success": true }

GET /api/crm/auth/me

Description: Returns the currently authenticated user from the session cookie.

Response (200): Same user object as login response above.

Error (401): { "error": "Not authenticated" }

5. Core CRM API Endpoints

All endpoints under `/api/crm/*` require authentication unless otherwise noted. The `crm_session` HTTP-only cookie must be sent with every request.

Public Endpoints (No Auth Required)

Method	Endpoint	Description
POST	<code>/api/contact</code>	Contact form submission (rate limited: 5/15min per IP)
POST	<code>/api/chat</code>	AI chat with lead capture (rate limited: 20/15min per IP)
POST	<code>/api/crm/auth/login</code>	User authentication

CRM Endpoints (Auth Required)

Method	Endpoint	Description
GET/POST	<code>/api/crm/contacts</code>	List / Create contacts
GET/PUT/ DEL	<code>/api/crm/contacts/[id]</code>	Read / Update / Delete contact
GET/POST	<code>/api/crm/companies</code>	List / Create companies
GET/PUT/ DEL	<code>/api/crm/companies/[id]</code>	Read / Update / Delete company
GET/POST	<code>/api/crm/deals</code>	List / Create deals
GET/PUT/ DEL	<code>/api/crm/deals/[id]</code>	Read / Update / Delete deal
GET	<code>/api/crm/deals/[id]/activities</code>	Deal activity history
GET/POST	<code>/api/crm/pipeline</code>	Pipeline stages and deals
GET/POST	<code>/api/crm/invoices</code>	List / Create invoices
GET/PUT/ DEL	<code>/api/crm/invoices/[id]</code>	Read / Update / Delete invoice
POST	<code>/api/crm/invoices/[id]/send</code>	Mark invoice as sent
POST	<code>/api/crm/invoices/[id]/mark-paid</code>	Mark invoice as paid
GET	<code>/api/crm/invoices/[id]/pdf</code>	Generate invoice PDF
POST	<code>/api/crm/invoices/[id]/payments</code>	Record a payment
GET/POST	<code>/api/crm/proposals</code>	List / Create proposals

Method	Endpoint	Description
GET/PUT/ DEL	/api/crm/proposals/[id]	Read / Update / Delete proposal
POST	/api/crm/proposals/[id]/send	Send proposal via email
POST	/api/crm/proposals/[id]/status	Update proposal status
GET/POST	/api/crm/tasks	List / Create tasks
GET/PUT/ DEL	/api/crm/tasks/[id]	Read / Update / Delete task
GET/POST	/api/crm/meetings	List / Create meetings
GET/PUT/ DEL	/api/crm/meetings/[id]	Read / Update / Delete meeting
POST	/api/crm/meetings/[id]/cancel	Cancel meeting
POST	/api/crm/meetings/[id]/complete	Complete meeting
GET/POST	/api/crm/activities	List / Create activities
GET/POST	/api/crm/notes	List / Create notes
GET/POST	/api/crm/workflows	List / Create workflows
GET/PUT/ DEL	/api/crm/workflows/[id]	Read / Update / Delete workflow
POST	/api/crm/workflows/trigger	Trigger a workflow
GET	/api/crm/workflows/executions	List workflow executions
POST	/api/crm/email	Send manual email (logged as activity)
GET	/api/crm/dashboard	Dashboard stats and KPIs
GET	/api/crm/stats	General CRM statistics
GET/POST	/api/crm/reports	List / Create reports
POST	/api/crm/reports/export	Export report data
GET/PUT/ DEL	/api/crm/reports/[id]	Read / Update / Delete report
GET	/api/crm/reports/dashboard	Dashboard report data
GET/POST	/api/crm/tags	List / Create tags
GET/POST	/api/crm/installers	List / Create installers
GET/PUT/ DEL	/api/crm/installers/[id]	Read / Update / Delete installer
GET	/api/crm/installers/stats	Installer statistics

Method	Endpoint	Description
GET/POST	/api/crm/leads	List / Create leads
GET/PUT/DEL	/api/crm/leads/[id]	Read / Update / Delete lead
GET	/api/crm/leads/[id]/activities	Lead activities
GET/POST	/api/crm/settings	Get / Update user settings
POST	/api/crm/settings/password	Change password
GET	/api/crm/financial	Financial overview
POST	/api/crm/ai	AI assistant endpoint
POST	/api/crm/call	Initiate phone call

Billing Endpoints

Method	Endpoint	Description
GET/POST	/api/crm/billing/plans	List / Subscribe to plans
POST	/api/crm/billing/checkout	Create Stripe checkout session
GET	/api/crm/billing/status	Get billing status
POST	/api/crm/billing/webhook	Stripe webhook handler
GET	/api/crm/billing/portal	Stripe customer portal URL

Google Calendar Endpoints

Meth od	Endpoint	Description
GET	/api/crm/calendar/google/auth-url	Google OAuth authorization URL
GET	/api/crm/calendar/google/callback	Google OAuth callback handler
GET	/api/crm/calendar/google/status	Google Calendar connection status
POST	/api/crm/calendar/google/disconnect	Disconnect Google Calendar
GET	/api/crm/calendar/google/sync	Sync Google Calendar events
POST	/api/crm/calendar/google/push-event	Push event to Google Calendar
GET	/api/crm/calendar/google/events	List Google Calendar events

Analytics Endpoint

Method	Endpoint	Description
GET	/api/crm/analytics/website	Website analytics data

6. Postmark Email Integration

The email system is implemented in `src/lib/postmark.ts`. It uses the native fetch API (no third-party SDK dependency) to communicate with the Postmark REST API.

Architecture

Sender: `hello@renewably.ie` (configurable via `FROM_EMAIL` env var)

Authentication: `POSTMARK_SERVER_TOKEN` env var

Graceful degradation: Returns a mock success response if the token is not configured, preventing crashes in development.

Core Functions

`sendEmail(options: SendEmailOptions): Promise<PostmarkResponse>`

Sends a raw HTML email via Postmark.

Option	Type	Required	Notes
<code>to</code>	string	Yes	Recipient email address
<code>cc</code>	string	No	CC recipients
<code>bcc</code>	string	No	BCC recipients
<code>replyTo</code>	string	No	Reply-to address
<code>subject</code>	string	Yes	Email subject line
<code>htmlBody</code>	string	Yes	HTML email body
<code>textBody</code>	string	No	Plain-text fallback body
<code>tag</code>	string	No	Postmark tag for categorization
<code>trackOpens</code>	boolean	No	Default: true
<code>trackLinks</code>	string	No	Default: "HtmlOnly"

`sendTemplate(options: SendTemplateOptions): Promise<PostmarkResponse>`

Sends an email using a Postmark template. Options: `to`, `cc?`, `bcc?`, `templateId` (int), `templateModel` (object), `tag?`, `trackOpens?`.

Specialized Functions

`sendContactNotification(data: ContactNotificationData)`

Sends a branded notification to hello@renewably.ie when the public contact form is submitted. Renders an HTML table layout with name, email, phone, company, and message fields. Tagged "contact-form".

sendWelcomeEmail(name: string, email: string)

Auto-reply sent to the contact form submitter. Branded email containing a 4-step onboarding process. Tagged "welcome-auto-reply".

sendProposalEmail(data: ProposalEmailData)

Sends a proposal to a customer. Displays proposal title, total investment (EUR formatted), and valid-until date. Includes an optional CTA button linking to the proposal view. Tagged "proposal-sent".

sendInvoiceEmail(data: InvoiceEmailData)

Sends an invoice to a customer. Displays invoice number, amount due (EUR formatted), and due date. Includes an optional CTA button. Tagged "invoice-sent".

Response Interface

```
interface PostmarkResponse { ErrorCode: number; Message: string; MessageID: string; SubmittedAt: string; To: string; }
```

7. Supabase Migration Guide

The CRM currently uses SQLite via Prisma. This section provides a practical guide for migrating to Supabase (PostgreSQL).

Why Supabase

PostgreSQL database — Production-grade relational database with full ACID compliance.

Built-in real-time subscriptions — Enable live CRM dashboard and pipeline updates via WebSocket.

Row-Level Security (RLS) — Multi-tenant data isolation at the database level.

Built-in auth — Can supplement or replace the current bcrypt session system.

Edge functions — Offload selected API routes to Deno-based edge runtime.

Automatic backups & scaling — Managed infrastructure with point-in-time recovery.

Step 1 — Create Supabase Project

Create a new project at supabase.com. Retrieve the project URL, anon key, and service role key from Project Settings → API.

Step 2 — Update Environment Variables

```
DATABASE_URL=postgresql://postgres:[PASSWORD]@db.[PROJECT].supabase.co:5432/postgres
NEXT_PUBLIC_SUPABASE_URL=https://[PROJECT].supabase.co NEXT_PUBLIC_SUPABASE_ANON_KEY=[ANON_KEY]
SUPABASE_SERVICE_ROLE_KEY=[SERVICE_ROLE_KEY]
```

Step 3 — Update Prisma Schema

Change the datasource provider in `schema.prisma`:

```
datasource db { provider = "postgresql" url = env("DATABASE_URL") }
```

Step 4 — Run Migration

```
npx prisma migrate dev --name migrate_to_supabase
```

Step 5 — Install Supabase Client

```
npm install @supabase/supabase-js
```

Step 6 — Create Supabase Client Utility

New file: `src/lib/supabase.ts`

```
import { createClient } from '@supabase/supabase-js' const supabaseUrl =
process.env.NEXT_PUBLIC_SUPABASE_URL! const supabaseAnonKey =
process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY! export const supabase = createClient(supabaseUrl,
supabaseAnonKey) // Admin client with service role (bypasses RLS) export const supabaseAdmin =
process.env.SUPABASE_SERVICE_ROLE_KEY ? createClient(supabaseUrl,
process.env.SUPABASE_SERVICE_ROLE_KEY) : supabase
```

Step 7 — Enable Row-Level Security (RLS)

Apply RLS policies for key tables to enforce multi-tenant isolation:

```
-- Example: Contacts table ALTER TABLE "Contact" ENABLE ROW LEVEL SECURITY; CREATE POLICY
"Admins and managers can view all contacts" ON "Contact" FOR SELECT USING ( EXISTS ( SELECT 1
FROM "Session" s JOIN "User" u ON u.id = s."userId" WHERE s.token =
current_setting('request.header.x-session-token', true) AND u.role IN ('admin', 'manager') ) );
```

Step 8 — Real-Time Subscriptions

Enable live updates for the dashboard and pipeline views:

```
// Subscribe to new deals in real-time const subscription = supabase .channel('deals-changes')
.on('postgres_changes', { event: '*', schema: 'public', table: 'Deal', }, (payload) => {
console.log('Deal changed:', payload) // Update UI state }) .subscribe()
```

Step 9 — Optional: Supabase Auth

Supabase Auth can supplement or replace the current bcrypt session system:

```
// Sign up a new user const { data, error } = await supabase.auth.signUp({ email:
'user@example.com', password: 'secure-password', }) // Sign in const { data, error } = await
supabase.auth.signInWithPassword({ email: 'user@example.com', password: 'secure-password', })
```

Migration Considerations

Case sensitivity: SQLite is case-insensitive for string comparisons; PostgreSQL is case-sensitive. Update queries accordingly (use `ILIKE` or `LOWER()`).

Boolean types: SQLite uses `INTEGER` for booleans; PostgreSQL uses native `BOOLEAN`. Prisma handles this, but raw SQL queries need updating.

ID generation: SQLite `AUTOINCREMENT` differs from PostgreSQL `SERIAL/GENERATED ALWAYS`. The CUID-based IDs in the current schema will work fine without changes.

Redis rate limiting: Can optionally be replaced with Supabase's `pg_net` extension or kept as-is for better performance.

8. Rate Limiting

The CRM uses a dual rate limiting system to protect both public-facing and authenticated endpoints.

Public APIs (`src/lib/rate-limit.ts`)

Redis-backed rate limiting with in-memory fallback for environments without Redis.

Endpoint	Limit	Window	Scope
POST /api/contact	5 submissions	15 minutes	Per IP address
POST /api/chat	20 messages	15 minutes	Per IP address

CRM APIs (`src/lib/crm-validation.ts`)

In-memory rate limiting for authenticated CRM endpoints.

Endpoint	Limit	Window	Scope
POST /api/crm/auth/login	5 attempts	1 minute	Per IP address
POST /api/crm/email	10 requests	1 minute	Per IP address (configurable)
All CRM endpoints	Configurable	Configurable	Via <code>checkApiRateLimit(key, { maxAttempts, windowMs })</code>

General CRM rate limit: All authenticated endpoints can be rate-limited using the `checkApiRateLimit(key, { maxAttempts, windowMs })` function, allowing per-endpoint customization.

9. Stripe Billing Integration

The billing system is implemented in `src/lib/stripe.ts`. It provides subscription management via Stripe Checkout Sessions and Customer Portal.

Features

Subscription checkout via Stripe Checkout Sessions

Customer portal for self-service billing management (upgrade, downgrade, cancel)

Webhook handler for subscription lifecycle events (checkout.session.completed, customer.subscription.updated, etc.)

Plan tiers: Starter, Pro, Enterprise (Price IDs configured via environment variables)

Key Functions

Function	Description
getStripe()	Returns a singleton Stripe instance. Initializes lazily on first call.
getOrCreateCustomer(params)	Creates or retrieves a Stripe Customer. Links to internal User record.
createCheckoutSession(params)	Creates a Stripe Checkout Session for subscription purchase.
cancelSubscription(subscriptionId)	Cancels a subscription at the end of the current billing period.
getSubscription(subscriptionId)	Retrieves current subscription details from Stripe.
verifyWebhook(rawBody, signature)	Verifies Stripe webhook signature using STRIPE_WEBHOOK_SECRET.
createPortalSession(params)	Creates a Stripe Customer Portal session for self-service billing.

Webhook Endpoint

POST /api/crm/billing/webhook — Receives and processes Stripe webhook events. Verifies the signature, parses the event payload, and updates internal records accordingly. Must be registered in the Stripe Dashboard as the webhook endpoint.

10. Key Libraries & Utilities

The following internal libraries power the CRM system. All are located under `src/lib/`.

File	Purpose
src/lib/db.ts	Prisma client singleton — ensures a single database connection across the application lifecycle.
src/lib/postmark.ts	Postmark email integration — raw fetch API client with <code>sendEmail</code> , <code>sendTemplate</code> , and specialized email functions.
src/lib/crm-session.ts	Authentication — session creation, validation, password hashing (bcryptjs), and cookie management.
src/lib/crm-auth.ts	Auth middleware — <code>requireAuth()</code> and <code>requireAdmin()</code> wrappers for protected API routes.

File	Purpose
src/lib/crm-validation.ts	Input validation — request sanitization, parameter validation, and CRM API rate limiting.
src/lib/crm-schemas.ts	Zod schemas — TypeScript-first request/response validation schemas for all CRM endpoints.
src/lib/rate-limit.ts	Public API rate limiting — Redis-backed with in-memory fallback; sliding window algorithm.
src/lib/redis.ts	Redis client — ioredis connection with environment-based URL configuration.
src/lib/stripe.ts	Stripe billing — subscription checkout, portal management, webhook verification, customer sync.
src/lib/logger.ts	Structured JSON logger — configurable log levels (debug/info/warn/error) with timestamps.
src/lib/format.ts	Formatting utilities — date formatting (relative, ISO), currency (EUR), number formatting.
src/lib/utils.ts	General utilities — shared helper functions used across the application.